

Transformer Architecture

pig7selene

September 5, 2026

Abstract

A decoder-only Transformer maps a batch of token IDs to vocabulary logits through a sequence of shape-preserving residual updates. This chapter gives the complete high-level computation graph before treating attention, positional encoding, normalization, and optimization in dedicated technical chapters. It defines the hidden-state and tensor-shape contracts of the model, contrasts Pre-Norm and Post-Norm block organization, and identifies the parameter, activation, and sequence-length terms that dominate later systems analysis.

1 Introduction

A decoder-only Transformer is often introduced as a stack of Self-Attention layers. That description is useful but incomplete. A language model has a stable external contract: it receives a tensor of token IDs and produces one vocabulary-sized vector of logits at every position. Attention is only one transformation in the path between those two objects. The remaining transformations determine where normalization acts, how information is accumulated, which tensor dominates activation memory, and how the output vocabulary is scored.

This chapter gives that path a precise, high-level form. The aim is not to rederive attention, positional encoding, normalization, or feed-forward networks in isolation. Instead, the chapter establishes the computation graph that makes those details meaningful: a sequence enters a residual stream, each block writes updates to that stream, and a final projection turns the resulting states into logits for an autoregressive language-model objective.

2 Decoder-Only Transformer Architecture

2.1 From Token IDs to Hidden States

Consider a batch of B token sequences, each represented to length T . Let

$$X \in \{0, \dots, |\mathcal{V}| - 1\}^{B \times T} \quad (1)$$

be the integer token-ID tensor, let d denote the model dimension, and let $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ be the token-embedding matrix. Embedding lookup produces a rank-three tensor

$$H^0 = E[X] \in \mathbb{R}^{B \times T \times d}. \quad (2)$$

The superscript identifies depth rather than a power. Thus H^0 is the sequence representation before the first Transformer block, while H^ℓ will denote the representation after block ℓ . If the architecture uses additive absolute positions, the input is instead $H^0 = E[X] + P$, where $P \in \mathbb{R}^{T \times d}$ is broadcast across the batch. Other positional schemes modify later computations rather than this initial tensor. In every case, the central object is the same: for batch item b and position t , $H_{b,t,:}^0$ is a length- d hidden-state vector.

The distinction between IDs and states is fundamental. X is discrete and has no useful local geometry: token ID 800 is not intrinsically closer to ID 801 than to ID 7. Each slice of H^0 is continuous and trainable. Subsequent layers can mix information across positions and features without changing the sequence length.

2.2 Stacked Blocks and Causal Processing

Let L be the number of Transformer blocks. A decoder-only model applies a sequence of functions $F_1, \dots, F_L$ to the input representation:

$$H^\ell = F_{\ell(H^{\ell-1})}, \quad \ell \in \{1, \dots, L\}. \quad (3)$$

Each F_ℓ preserves the shape $B \times T \times d$. The expression **decoder-only** refers to the causal dependency pattern within these functions: the state at position t may draw on positions at most t , but not on later tokens. It does not mean that the block processes positions one at a time during training. The full tensor is usually evaluated in parallel under a causal Attention Mask. The sequential aspect of generation emerges only when the model is used to append a new token to an existing prefix.

The original Transformer paired an encoder stack with a decoder stack for sequence-to-sequence tasks [1]. A Foundation Model used for Causal Language Modeling retains the masked, autoregressive branch and omits encoder-decoder cross-attention. This reduction creates a uniform contract: a single stream of token positions is transformed repeatedly, then scored against the vocabulary. GPT-2 is an influential example of this decoder-only pattern [2].

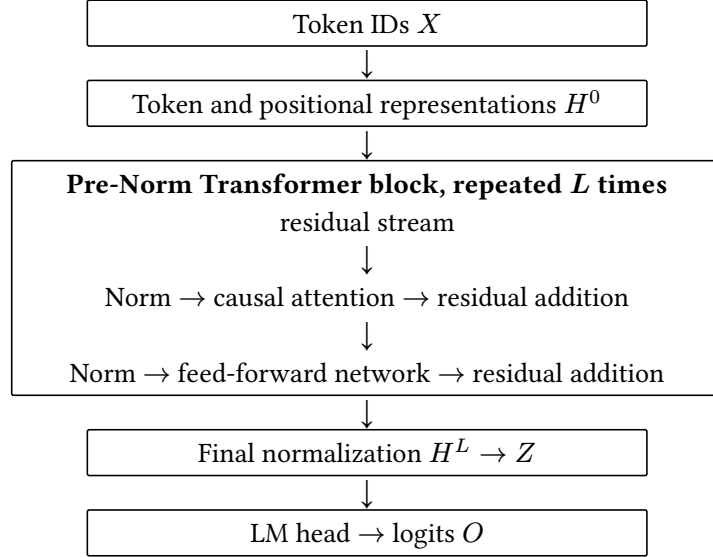


Figure 1: A high-level decoder-only Transformer computation path. Each block reads the current residual stream and writes an update to it.

The diagram in Figure 1 omits internal parameter matrices on purpose. It exposes the architectural invariant that matters at this stage: every block receives a hidden-state tensor and returns a tensor of the same shape. Depth is therefore a controlled composition of representation updates rather than a sequence of incompatible interfaces.

3 Transformer Block

A standard decoder-only block has two learned sublayers: causal Self-Attention and a position-wise feed-forward network (FFN). Attention allows the representation at one position to use an allowed prefix of the sequence; the FFN applies the same nonlinear transformation independently at every position. Residual connections preserve a direct route around both sublayers, while normalization controls the scale and distribution of the vectors passed into them.

For a Pre-Norm block, write A_ℓ for the attention transformation, M_ℓ for the FFN transformation, and N_ℓ^a and N_ℓ^m for their respective normalizations. The two updates are

$$U^\ell = H^{\ell-1} + A_{\ell(N_\ell^a(H^{\ell-1}))}, \quad (4)$$

followed by

$$H^\ell = U^\ell + M_{\ell(N_\ell^m(U^\ell))}. \quad (5)$$

These equations describe control flow, not an attempt to hide the details in notation. A_ℓ contains the query, key, value, mask, softmax, and output operations. M_ℓ contains the expansion, activation or

gate, and contraction. Their outputs have the same final feature dimension d as their inputs, so they can be added to the residual stream.

The order of the two sublayers is conventional rather than mathematically necessary. The important point is that attention and the FFN have complementary roles. Attention mixes **positions**: a token representation can incorporate information from its prefix. The FFN mixes **features** within one position: it turns the resulting feature vector into a richer nonlinear update. A model needs both forms of computation to make context-sensitive predictions without treating every feature interaction as an attention operation.

4 Residual Stream

The phrase **residual stream** is a useful abstraction for H^ℓ . At any depth, it is the common sequence representation carried through the network. Attention does not replace that representation; it reads a normalized view of it and adds a context-dependent update. The FFN then reads the updated stream and adds a position-wise update. The paired updates in Equation 4 and Equation 5 therefore describe an additive computation in which information can persist across many layers while new features are written into the same width- d state.

This view has two benefits. First, it makes tensor shapes easy to audit: the residual stream remains $B \times T \times d$ from input to final normalization. Second, it discourages a misleading picture in which a Transformer repeatedly discards and reconstructs a sequence. Each block instead makes bounded modifications to a shared representation. The identity terms in the residual connections provide direct forward and backward routes, while the learned sublayers specialize in producing useful corrections.

5 Pre-Norm and Post-Norm

The original Transformer uses a Post-Norm organization, in which normalization follows each residual addition. For a generic sublayer S , that pattern is

$$y = N(x + S(x)). \quad (6)$$

By contrast, Pre-Norm applies normalization to the sublayer input and leaves the residual path itself unnormalized:

$$y = x + S(N(x)). \quad (7)$$

Neither expression is merely a rearrangement of the other. In Post-Norm, the next sublayer receives a normalized mixture of the previous stream and the current update. In Pre-Norm, the identity path from x to y remains explicit, while the learned update is driven by a normalized argument. Consequently, normalization placement changes both forward activation statistics and the paths followed by gradients.

This distinction became practically consequential as Transformer stacks grew deeper. Xiong et al. analyze the initialization behavior of Pre-Norm and Post-Norm Transformers and relate Post-Norm’s output-layer gradient scale to the usefulness of warmup [3]. Modern decoder-only architectures commonly adopt a Pre-Norm family of designs; GPT-2, for example, places LayerNorm at the input of each sub-block [2]. These observations do not make Pre-Norm universally superior. They identify a stability trade-off whose interaction with optimizer choice, residual scaling, precision, and depth must be evaluated as a system.

6 From Hidden States to Logits

After the final block, many decoder-only models apply a final normalization N_f :

$$Z = N_f(H^L), \quad Z \in R^{B \times T \times d}. \quad (8)$$

The language-model head is an affine map from each final hidden vector to one score per vocabulary item. With output matrix $W_{\text{out}} \in R^{d \times |\mathcal{V}|}$ and bias $b_{\text{out}} \in R^{|\mathcal{V}|}$,

$$O = ZW_{\text{out}} + b_{\text{out}}, \quad O \in R^{B \times T \times |\mathcal{V}|}. \quad (9)$$

The element $O_{b,t,v}$ is a logit, not yet a probability: it is the unnormalized score assigned to vocabulary item v at position t . An autoregressive objective applies a masked cross-entropy loss to these scores. At inference time, a decoding rule converts the final position’s logits into a distribution or selected next token.

When input and output embeddings are tied, the model reuses the token-embedding table and takes $W_{\text{out}} = E^T$, up to the orientation convention of the implementation. Weight tying reduces parameter count and connects the geometry used to read a token with the geometry used to score it. It is an architectural choice, not an identity required by decoder-only modeling. The original Transformer discusses shared embedding and pre-softmax weights in some configurations [1].

7 Tensor Shapes Through the Model

The following table records the shape contracts most useful when implementing or debugging the high-level forward pass. Let h be the number of query heads, d_h the per-head dimension, and d_{ff} the FFN intermediate width; ordinarily $d = hd_h$.

Quantity	Shape	Role
Token IDs X	$B \times T$	Discrete input indices.
Embeddings / residual stream H^ℓ	$B \times T \times d$	Sequence representation at depth ℓ .
Queries, keys, values	$B \times T \times h \times d_h$	Head-partitioned attention inputs.
Attention output	$B \times T \times d$	Contextual update projected back to model width.
FFN intermediate	$B \times T \times d_{\text{ff}}$	Expanded position-wise representation.
Logits O	$B \times T \times \mathcal{V} $	Vocabulary scores for the language-model objective.

Table 1: Symbolic tensor shapes in a decoder-only Transformer. The exact internal layout of heads may vary by implementation, but the residual-stream and logit contracts are stable.

The table distinguishes model-width tensors from vocabulary-width tensors. Most block activations have width d , whereas logits have width $|\mathcal{V}|$. For a large vocabulary, materializing logits for every position can itself be substantial. Conversely, the residual stream persists at every block and is a major source of activation memory during training. Shape reasoning is therefore not bookkeeping after the fact; it is the first step toward a reliable memory and compute model.

8 Computational Structure

At a high level, each block contains four dense projection families associated with attention: query, key, value, and output. In a conventional full-width implementation, they contribute on the order of $4d^2$ parameters per layer. The FFN contributes roughly $2dd_{\text{ff}}$ parameters for a two-projection design, or a different constant factor for gated variants such as SwiGLU. These counts are only first-order approximations, but they already explain why changing d , L , or d_{ff} changes the parameter budget materially.

The principal sequence-length dependence arises in attention. For a length- T sequence, each position can compare against a prefix whose total pairwise work grows on the order of T^2d for dense Self-Attention. The FFN instead performs work on each position independently, scaling on the order of Tdd_{ff} . The relative bottleneck therefore changes with sequence length, width, batch size, hardware, and kernel implementation. Cache-aware decoding and other inference methods alter the execution regime without changing the basic architectural contract.

Activation memory has a different emphasis from parameter count. During training, backward computation requires access to selected forward intermediates or to recomputed equivalents. The residual stream alone stores BTd scalars per layer before accounting for attention and FFN internals. During autoregressive decoding, the model instead retains key and value representations for prior positions in a KV Cache. The forward equations are the same in spirit, but the dominant persistent state changes with the execution regime.

9 Summary

A decoder-only Transformer maps token IDs to embeddings, repeatedly applies residual attention and FFN updates, normalizes the final stream, and projects each hidden state to vocabulary logits. The stack in Equation 3 supplies the architectural spine; the Pre-Norm updates in Equation 4 and Equation 5 make the residual stream explicit; and the LM head in Equation 9 closes the path back to the discrete vocabulary.

The high-level view deliberately treats causal attention, positional encoding, normalization, and optimization as modules. A subsequent independent chapter can derive attention masks, head variants, and positional encodings; another can study FFNs, normalization, residual scaling, and gradient flow. The computation graph developed here supplies the common tensor contracts that those analyses require.

References

- [1] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners,” technical report, 2019. [Online]. Available: <https://cdn.openai.com/better-language-models/language-models.pdf>
- [3] R. Xiong *et al.*, “On Layer Normalization in the Transformer Architecture,” in *Proceedings of the 37th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 10524–10533.